

Department of Computer Science

Moderne Softwarearchitektur

MSA26-PeerReview-Gruppe-A

Abgabedatum:

20.06.2026

Eingereicht von:

Matthias Matthies

Matr.-Nr.: 106971
stud106971@fh-wedel.de

Erika Musterfrau

Matr.-Nr.: 654321
stud654321@fh-wedel.de

Max Mustermann

Matr.-Nr.: 123456
stud123456@fh-wedel.de

John Doe

Matr.-Nr.: 987654
stud987654@fh-wedel.de

Betreut von:

Prof. Dr. Ulrich Hoffmann

Fachhochschule Wedel
Feldstraße 143
22880 Wedel
Tel: 04103 - 8048 - 41
urlic.hoffmann@fh-wedel.de

Table of Content

1	Introduction (Done for now)	1
2	Foundations of Logical Time (Todo: Write + Translate + Rewrite)	2
2.1	Partial Ordering: Happenes-before (Todo: Translate + Rewrite)	2
2.2	Logical Clocks (Todo: Write + Translate + Rewrite)	3
2.3	State Machine Replication & Consensus (Todo: Write + Translate + Rewrite) .	3
3	Modern Applications of Event Ordering	4
4	Happens-Before in Static Code Anaylsis	5
5	Summary	6
5.1	Conclusion	6
5.1.1	Thema #1	6
5.1.2	Thema #2	6
A	Appendix	7
A.1	Unterteilung der Annotations-Attribute	7
	Bibliography	8
	Declaration of Authorship	9

1 Introduction (Done for now)

As fields like artificial intelligence emerge and the broader landscape of computer science grows increasingly complex, one can easily become entangled in hyper-specific sub-disciplines. Therefore, it is essential to take a step back and revisit the foundational papers that have shaped the field.

Almost half a century after the release of his seminal paper, „Time, Clocks, and the Ordering of Events in a Distributed System“ [1], Leslie Lamport’s work continues to provide a vital foundation for a wide variety of domains across computer science. This seminar paper first analyze Lamport’s original publication. This work then sheds light on several modern computer science disciplines where Lamport’s ideas remain highly influential. Afterwards, the field of static code analysis is explored in depth to further demonstrate the reach of his work. This paper concludes with a summary that underscores the ongoing significance of these concepts, alongside some concluding remarks.

2 Foundations of Logical Time (Todo: Write + Translate + Rewrite)

This chapter will contain a throughout explanation of Lamport's concepts published in the original paper, „Time, Clocks, and the Ordering of Events in a Distributed System“ [1]. It will be split into three main topics, the partial ordering „happenes-before“, logical clocks and state machine replication.

2.1 Partial Ordering: Happenes-before (Todo: Translate + Rewrite)

The „happenes-before“ relation is a relation to order an event a and an event b

Die happens-before Relation definiert unabhängig der physischen Zeit eine Ordnung für das Eintreten von events a und b in einem System. Definiert wird die Relation wie folgt:

1. Sind a und b events im gleichen System, und a tritt vor b ein, dann gilt $a \rightarrow b$.
2. Wenn a das Senden einer Nachricht von einem Prozess und b das Empfangen der Nachricht in einem anderen Prozess ist, dann gilt $a \rightarrow b$.
3. Wenn $a \rightarrow b$ und $b \rightarrow c$, dann gilt auch $a \rightarrow c$. Zwei unterschiedliche Events a und b treten zeitgleich ein, gdw. $a \not\rightarrow b$ und $b \not\rightarrow a$.

Außerdem wird angenommen, für jedes Event a gilt $a \not\rightarrow a$. Die relation happens-before ist also eine irreflexive partial ordering on the set of all events in the system [1, p. 559].

Zur Veranschaulichung führt Lamport die in Abbildung 1 und Abbildung 2 zu findenden folgenden Grafiken ein. Das „Space-time diagram“ [1, p. 559], zeigt wie die Events in Verschiedener Prozesse oder Systeme zusammenhängen. Abbildung 1 zeigt drei Prozesse (P, Q und R) und deren Events ($p_1, \dots, p_4, q_1, \dots, q_7$ und $r_1, \dots, r_4$). Die vertikalen geraden Linien repräsentieren einen zeitlichen Verlauf eines Prozess und die „wavy lines“ repräsentieren das Versenden von Nachrichten unter den Prozessen. Hierzu gilt $p_i \rightarrow p_{i+1}, q_i \rightarrow q_{i+1}$ und $r_i \rightarrow r_{i+1}$. Der rechte Teil der Abbildung zeigt die selben Prozesse und Events in einer entzerrten, aufgeteilten Ansicht. Die Semantik der Events und Messages bleibt hierbei unverändert, es ist nun aber einfacher zu erkennen, für welche Events zwischen den einzelnen Prozessen ein „happenes-before“ gilt. Lamport erwähnt außerdem, dass die happens-before Relation auch so verstanden werden kann, dass gilt: $a \rightarrow b \Rightarrow a$ can causally affect b . Andernfalls gelten die Events als concurrent.

Abbildung 1: Space time diagram

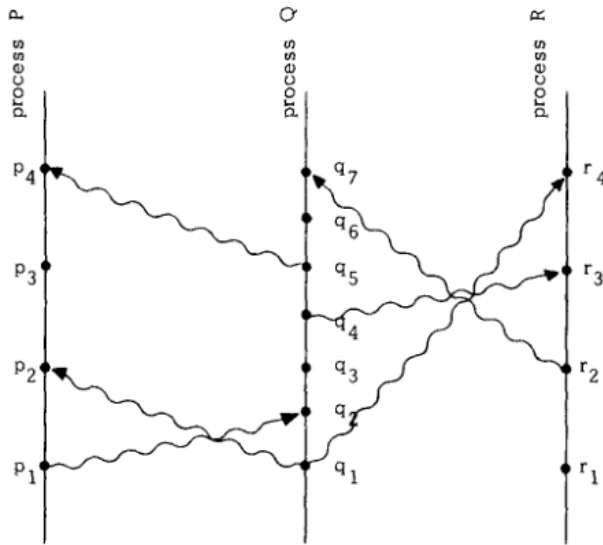
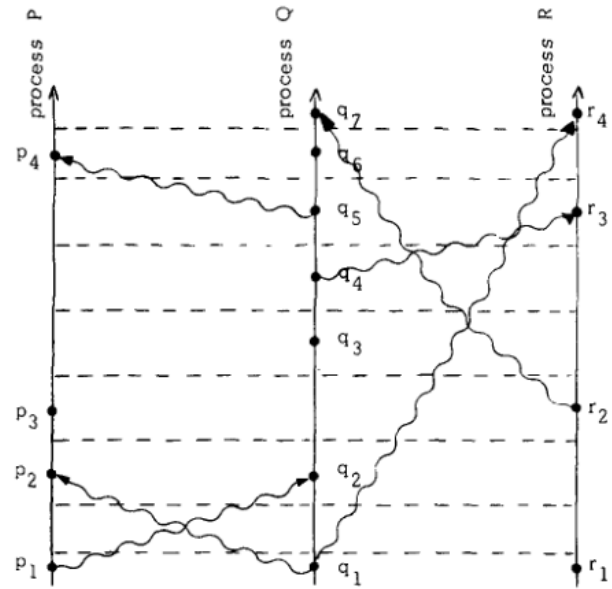


Abbildung 2: Space time diagram with logical time



2.2 Logical Clocks (Todo: Write + Translate + Rewrite)

Aufbauend auf der happens-before Relation führt Lamport die Logical Clocks ein. Für jeden Prozess P_i wird eine Clock C_i als eine Funktion definiert, welche jedem Event a einer Zahl zuordnet: $C_i\langle a \rangle$. Außerdem werden alle Clocks als eine Funktion C definiert, welche jedem Event b die Nummer $C\langle b \rangle = C_j\langle b \rangle$ zuordnet, wenn b ein Event in Prozess P_j ist. Wichtig ist, dass diese Zahlen $C_i\langle a \rangle$ in keinem Zusammenhang mit der physischen Zeit stehen, es ist eher eine abstrakte oder logische Zeiteinheit. Diese Zeiteinheiten können beispielsweise die „dotted lines“ in Abbildung 1 sein. Um nun die Korrektheit eines Systems aus clocks zu definieren wird die Clock Condition eingeführt. Diese besagt, dass für jedes event a, b gilt: wenn $a \rightarrow b$ dann $C\langle a \rangle < C\langle b \rangle$. The converse condition is not true, as Abbildung 1 shows p_2 and p_3 are both concurrent with q_3 , which would in turn mean they happen at the same time, but $p_2 \rightarrow p_3$ contradicts the concurrency of p_2 and p_3 .

If the following conditions hold:

1. If a and b are events in process P_i and a comes before b , then $C_i\langle a \rangle < C_i\langle b \rangle$
2. If a is the sending of a message by process P_i and b is the receipt of that message by process P_j , then $C_i\langle a \rangle < C_j\langle b \rangle$

the clock condition is satisfied [1, p. 560].

2.3 State Machine Replication & Consensus (Todo: Write + Translate + Rewrite)

3 Modern Applications of Event Ordering

4 Happens-Before in Static Code Analysis

5 Summary

5.1 Conclusion

Proin gravida dictum faucibus. Sed at semper lectus. Nam ultrices volutpat porta. Nulla non eros sagittis, sodales diam eget, cursus arcu. Integer ipsum metus, convallis a felis in, convallis aliquet urna. Morbi vel odio sed magna aliquam finibus. Proin blandit quis est ac mollis.

5.1.1 Thema #1

Integer venenatis convallis erat, sit amet elementum odio egestas interdum. Phasellus sagittis vestibulum libero vel sodales. Phasellus a facilisis felis, vel mollis velit. Quisque pulvinar turpis vitae bibendum condimentum. Nulla facilisi. Praesent iaculis euismod eros et consequat. Nullam efficitur facilisis sapien. Aenean non laoreet lectus. Phasellus tellus lorem, ullamcorper vel aliquam ut, sollicitudin accumsan nibh.

5.1.2 Thema #2

Vivamus quis lacus quis urna suscipit auctor. Nunc dignissim massa eu leo condimentum euismod. Nunc mattis quis mauris a pharetra. Duis eu iaculis magna, id tristique dui. Duis sagittis mauris id elit maximus auctor. Pellentesque vitae ipsum elementum, iaculis enim mollis, fringilla sem.

A Appendix

A.1 Unterteilung der Annotations-Attribute

Bibliography

- [1] L. Lamport, „Time, Clocks, and the Ordering of Events in a Distributed System“, Bd. 21, Nr. 7, 1978.

Declaration of Authorship

I hereby declare... (Add your full text here).

Matthias Matthies

Erika Musterfrau

Max Mustermann

John Doe